

# Integrating Temporal Memory into GraphCast via Stateful Mamba:

## Experiment Report and Architectural Analysis

Lianghong Mo

April 4, 2026

### Abstract

We report on a series of experiments integrating a Mamba-style selective state space model (SSM) into the GraphCast weather forecasting architecture. The goal is to provide GraphCast with *temporal memory* across autoregressive rollout steps, allowing the model to retain information from earlier in a forecast trajectory rather than relying solely on the two most recent input frames. We describe three architectural variants developed iteratively, identify critical bugs and design flaws in the first two, and propose a corrected *residual memory* architecture that isolates the contribution of temporal memory from confounding factors. We provide complete experimental results for the first two phases and the design rationale for the third.

## Contents

|          |                                                              |          |
|----------|--------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction and Motivation</b>                           | <b>2</b> |
| <b>2</b> | <b>GraphCast Architecture Overview</b>                       | <b>2</b> |
| <b>3</b> | <b>Architectural Variants</b>                                | <b>2</b> |
| 3.1      | Version 1: Stateless Mamba . . . . .                         | 2        |
| 3.2      | Version 2: Stateful Mamba . . . . .                          | 3        |
| 3.3      | Version 3: Residual Memory (Current) . . . . .               | 4        |
| <b>4</b> | <b>Cross-Rollout State Flow</b>                              | <b>5</b> |
| <b>5</b> | <b>Experimental Results</b>                                  | <b>6</b> |
| 5.1      | Common Parameters . . . . .                                  | 6        |
| 5.2      | Phase 1: Stateless Mamba . . . . .                           | 6        |
| 5.3      | Phase 2: Baseline Reference (Fair Comparison Grid) . . . . . | 6        |
| 5.4      | Phase 2: Stateful Mamba (Old Encoding Path) . . . . .        | 7        |
| <b>6</b> | <b>Diagnosis: Why Previous Approaches Failed</b>             | <b>7</b> |
| <b>7</b> | <b>Phase 3: Residual Memory Experiments</b>                  | <b>8</b> |
| 7.1      | Experimental Design . . . . .                                | 8        |
| 7.2      | Interpretation Guide . . . . .                               | 8        |
| 7.3      | Status . . . . .                                             | 8        |

|           |                                                                           |          |
|-----------|---------------------------------------------------------------------------|----------|
| <b>8</b>  | <b>Discussion</b>                                                         | <b>8</b> |
| 8.1       | Why Residual Memory Is the Right Formulation . . . . .                    | 8        |
| 8.2       | Open Questions . . . . .                                                  | 8        |
| 8.3       | Computational Considerations . . . . .                                    | 9        |
| <b>9</b>  | <b>Conclusion</b>                                                         | <b>9</b> |
| <b>10</b> | <b>Why Mamba May Not Surpass the Baseline</b>                             | <b>9</b> |
| 10.1      | The Baseline Is Already Surprisingly Effective . . . . .                  | 9        |
| 10.2      | Information Bottleneck: 128 Dimensions for an Entire Atmosphere . . . . . | 10       |
| 10.3      | The Forgetting Problem . . . . .                                          | 10       |
| 10.4      | Optimization Difficulty of Long Rollouts . . . . .                        | 10       |
| 10.5      | The Core Problem: Marginal Value . . . . .                                | 10       |

# 1 Introduction and Motivation

GraphCast [1] is a graph neural network (GNN) based weather forecasting model that predicts the next atmospheric state from the two most recent time steps. For multi-day forecasts, predictions are fed back autoregressively:

$$(\mathbf{x}_{t-1}, \mathbf{x}_t) \xrightarrow{\text{GraphCast}} \hat{\mathbf{x}}_{t+1} \xrightarrow{\text{feed back}} (\mathbf{x}_t, \hat{\mathbf{x}}_{t+1}) \xrightarrow{\text{GraphCast}} \hat{\mathbf{x}}_{t+2} \rightarrow \dots$$

Each prediction step is **memoryless**: the model sees only the two most recent frames and has no access to earlier atmospheric states. This limits its ability to:

- Track slowly evolving large-scale patterns (planetary waves, blocking events)
- Correct for systematic drift during long rollouts
- Leverage the temporal coherence inherent in atmospheric dynamics

Mamba [2], a selective state space model, provides a natural mechanism for temporal memory. Its hidden state  $h_t$  accumulates information over time with learned forgetting:

$$h_t = \bar{A}_t \cdot h_{t-1} + (1 - \bar{A}_t) \cdot u_t, \quad \bar{A}_t = \exp(A \cdot \Delta_t) \quad (1)$$

$$y_t = h_t \odot \sigma(g_t) + s \odot u_t \quad (2)$$

where  $\Delta_t = \text{softplus}(\text{Linear}(u_t))$  is a data-dependent discretization step,  $g_t$  is a gating signal, and  $s$  is a learnable skip connection. If this state can be carried across autoregressive rollout steps, it gives the model a persistent summary of the entire forecast trajectory.

## 2 GraphCast Architecture Overview

GraphCast operates on an icosahedral mesh as an intermediate representation. The pipeline for a single prediction step is:

1. **Feature Extraction**: All input time steps are concatenated along the channel dimension, producing a single feature vector per grid node:  $[\mathbf{x}_{t-1}; \mathbf{x}_t] \in \mathbb{R}^{2C}$ .
2. **Grid-to-Mesh Encoder**: A GNN maps grid node features to mesh node features via a single message-passing step, producing  $\mathbf{M} \in \mathbb{R}^{N_{\text{mesh}} \times B \times D}$ .
3. **Mesh Processor**: Multiple rounds of message passing on the icosahedral mesh graph enable global spatial communication.
4. **Mesh-to-Grid Decoder**: A GNN maps updated mesh features back to the latitude-longitude grid.
5. **Output**: The grid features are converted to physical variable predictions.

A critical observation: the channel concatenation in step 1 allows the Grid-to-Mesh GNN to *jointly* process both time steps in a single MLP pass, implicitly learning temporal features such as velocity ( $\mathbf{x}_t - \mathbf{x}_{t-1}$ ) and local tendencies. This is an efficient and effective encoding of short-term temporal information.

The autoregressive wrapper (`autoregressive.py`) uses `hk.scan` to loop the single-step predictor over multiple target steps. Importantly, `hk.scan` (unlike `jax.lax.scan`) automatically threads Haiku state—including `hk.get_state/hk.set_state` values—between iterations.

## 3 Architectural Variants

We developed three variants iteratively, each addressing problems found in the previous one.

### 3.1 Version 1: Stateless Mamba

File: `src/models/temporal_mesh_mamba.py`

**Design:** Replace the channel-concatenation encoding with per-timestep encoding. Each input frame is encoded independently through the Grid-to-Mesh GNN, producing  $\mathbf{M} \in \mathbb{R}^{T \times N_{\text{mesh}} \times B \times D}$ . A Mamba block then processes the time axis:

1. Transpose to  $(B \cdot N_{\text{mesh}}, T, D)$
2. Apply SSM scan along  $T$  with  $h_0 = \mathbf{0}$
3. Take last timestep output  $\rightarrow (N_{\text{mesh}}, B, D)$
4. Pass to Mesh GNN

The hidden state resets to zeros every forward pass—no memory across rollout steps.

**Problems identified:**

1. **Transpose bug:** The output of `_run_sequence` had an erroneous transpose that swapped the mesh-node and batch dimensions:

```
# BUG: extra transpose swaps [mesh, batch, D] to [batch, mesh, D]
return jnp.transpose(sequence[-1], (1, 0, 2))

# FIX: sequence[-1] is already [mesh, batch, D]
return sequence[-1]
```

This meant all stateless Mamba outputs had corrupted dimension ordering. The downstream GNN was robust enough to partially compensate, so the model still trained to similar loss—but the Mamba block’s output was effectively meaningless.

2. **Encoding path change:** By switching from channel concatenation to per-timestep encoding, we inadvertently weakened the encoder. The baseline’s joint processing of both frames in a single MLP pass is more expressive than independent per-frame encoding followed by temporal fusion.
3. **Redundant processing:** The Mamba block processes 2–8 frames that have already been individually encoded. The GNN encoder can already extract temporal features from concatenated channels. Mamba adds parameters and optimization complexity without providing new information.

### 3.2 Version 2: Stateful Mamba

**File:** `src/models/temporal_mesh_mamba_stateful.py`

**Design:** Same as Version 1, but the SSM hidden state persists across autoregressive rollout steps via Haiku’s state mechanism:

```
# Load state from previous rollout step (or zeros at sample start)
stored = hk.get_state("ssm_state", shape=(n_mesh, H), init=jnp.zeros)
init_state = jnp.tile(stored, (batch_size, 1))

# SSM scan over input timesteps
final_state, outputs = jax.lax.scan(step_fn, init_state, ...)

# Save for next rollout step (average over batch dimension)
hk.set_state("ssm_state", final_state.reshape(batch, mesh, H).mean(0))
```

The training loop resets SSM state to zeros before each training sample via `_reset_ssm_state()`, preventing cross-sample leakage. Within a sample, the `hk.scan` in `autoregressive.py` threads the state across rollout steps.

**Key design decisions:**

- **State shape**  $(N_{\text{mesh}}, H)$ : Independent of batch size to avoid shape mismatches between `hk.init` (batch=1) and `hk.apply` (batch>1).

- **Batch averaging:** Per-batch states are averaged before storage, creating a “consensus” state across different time windows in the batch.

**Problem:** The transpose bug was fixed, but the **encoding path difference persisted**. Stateful Mamba still used per-timestep encoding while the baseline used channel concatenation. Any performance comparison was confounded by two simultaneous changes: (1) the Mamba module itself, and (2) the weaker encoding path. It was impossible to attribute performance differences to temporal memory alone.

Additionally, with `target_steps` of only 2 or 4, the rollout was too short for the hidden state to accumulate meaningful long-term information beyond what the 2-frame input already provided.

### 3.3 Version 3: Residual Memory (Current)

**Files:** `temporal_mesh_mamba_stateful.py` (modified) + `graphcast.py` (new location)

**Design principle:** Use the *exact same encoding path* as baseline, then inject cross-rollout temporal memory as a pure residual. The **only** difference from baseline is the Mamba residual block.

1. Channel concatenation  $[\mathbf{x}_{t-1}; \mathbf{x}_t]$  (same as baseline)
2. Grid-to-Mesh GNN  $\rightarrow \mathbf{M} \in \mathbb{R}^{N_{\text{mesh}} \times B \times D}$  (same as baseline)
3. **Mamba residual block:** Unsqueeze to  $\mathbb{R}^{1 \times N_{\text{mesh}} \times B \times D}$ , run SSM (1 step + hidden state), squeeze back. Output:  $\mathbf{M}' = \mathbf{M} + \text{proj}(\text{Mamba}(\mathbf{M}, h_{\text{prev}}))$  (**new**)
4. Mesh GNN processor (same as baseline)
5. Mesh-to-Grid decoder (same as baseline)

**Code change in `graphcast.py`:**

```
if (self._temporal_backbone == "none"
    or self._temporal_location == "mesh_post_encoder_residual"):
    # Baseline encoding: concatenate channels, single Grid2Mesh pass
    grid_node_features = self._inputs_to_grid_node_features(inputs, forcings)
    (latent_mesh_nodes, latent_grid_nodes
     ) = self._run_grid2mesh_gnn(grid_node_features)
    # NEW: inject cross-rollout memory via stateful Mamba residual
    if self._temporal_location == "mesh_post_encoder_residual":
        latent_mesh_nodes = self._run_temporal_mesh_block(
            latent_mesh_nodes, is_training=is_training)
```

**Code change in `temporal_mesh_mamba_stateful.py`:**

```
def __call__(self, mesh_latent_tnbd, *, is_training):
    if mesh_latent_tnbd.ndim == 3:
        # [mesh, batch, D] -> unsqueeze time=1 -> Mamba (1 step + state) -> squeeze
        mesh_4d = mesh_latent_tnbd[None] # [1, mesh, batch, D]
        out_4d = self._run_sequence(mesh_4d, is_training=is_training)
        return out_4d[0] # [mesh, batch, D]
```

**Properties:**

- **Fair comparison:** Encoding path is identical to baseline. The only variable is the Mamba block.
- **Cannot degrade below baseline:** Residual connection means the worst case is learning zero weights (identity function).
- **Right task for Mamba:** Instead of redundantly processing 2–4 input frames, Mamba focuses on its actual strength—accumulating information across rollout steps via hidden state.
- **Faster:** No per-timestep encoding overhead. Each forward pass is baseline speed plus one small Mamba block.

## 4 Cross-Rollout State Flow

The stateful Mamba hidden state is threaded across autoregressive rollout steps via three mechanisms:

1. `hk.scan in autoregressive.py`: Threads all Haiku state (including `hk.get_state/hk.set_state`) between iterations. No modification needed.
2. `hk.get_state/hk.set_state in _StatefulSSMBlock`: Each rollout step loads the previous state, runs the SSM, and saves the new state.
3. `_reset_ssm_state in training loop`: Zeros all SSM states before each training sample to prevent cross-sample leakage.

For a training sample with `target_steps=6`:

Sample start  $\rightarrow h = \mathbf{0}$

Step 1: input =  $[t-1, t]$   $\xrightarrow{\text{encode}} \mathbf{M}_1 \xrightarrow{\text{Mamba}(h=\mathbf{0})} \mathbf{M}'_1, h_1 \rightarrow \hat{\mathbf{x}}_{t+1}$

Step 2: input =  $[t, \hat{\mathbf{x}}_{t+1}] \xrightarrow{\text{encode}} \mathbf{M}_2 \xrightarrow{\text{Mamba}(h_1)} \mathbf{M}'_2, h_2 \rightarrow \hat{\mathbf{x}}_{t+2}$

$\vdots$

Step 6: input =  $[\hat{\mathbf{x}}_{t+4}, \hat{\mathbf{x}}_{t+5}] \xrightarrow{\text{encode}} \mathbf{M}_6 \xrightarrow{\text{Mamba}(h_5)} \mathbf{M}'_6, h_6 \rightarrow \hat{\mathbf{x}}_{t+6}$

Next sample  $\rightarrow h = \mathbf{0}$

At step  $k$ , the hidden state  $h_k$  contains a compressed summary of the atmospheric evolution from the initial conditions through all  $k$  prediction steps—information that is unavailable to the memoryless baseline.

The loss is the mean of per-step weighted MSE across all target steps:

$$\mathcal{L} = \frac{1}{K} \sum_{k=1}^K \text{wMSE}(\hat{\mathbf{x}}_{t+k}, \mathbf{x}_{t+k})$$

Gradients flow back through all  $K$  steps. When  $K > 1$ , `hk.remat` (gradient checkpointing) is applied automatically to manage memory.

## 5 Experimental Results

### 5.1 Common Parameters

Table 1: Shared experimental configuration

| Parameter             | Value                                              |
|-----------------------|----------------------------------------------------|
| Base model            | GraphCast_small                                    |
| Pretrained checkpoint | ERA5 1979–2015, resolution 1.0°, mesh 2–5          |
| Training data         | ERA5 (WeatherBench2), 2020–2021, 6h intervals      |
| Evaluation data       | ERA5, 2022                                         |
| Variables             | 6 atmospheric + 5 surface, 13 pressure levels      |
| Optimizer             | AdamW (lr = $10^{-4}$ , weight_decay = $10^{-4}$ ) |
| Precision             | bfloat16                                           |
| GPU                   | NVIDIA A100 80GB SXM4                              |
| Mamba hidden size     | 128                                                |
| Mamba layers          | 1                                                  |
| Mamba dropout         | 0.0                                                |

### 5.2 Phase 1: Stateless Mamba

Table 2: Phase 1 configuration: mesh\_size=3, batch\_size=4, target\_steps=1, 20k steps

| Model                         | RMSE @8k      | MAE @8k      | vs. Baseline |
|-------------------------------|---------------|--------------|--------------|
| Baseline (input_steps=2)      | <b>138.55</b> | <b>35.13</b> | —            |
| Mamba h2 (stateless, input=2) | 139.39        | 35.47        | +0.6%        |
| Mamba h4 (stateless, input=4) | 138.83        | 35.39        | +0.2%        |
| Mamba h6 (stateless, input=6) | 139.47        | 35.39        | +0.7%        |
| Mamba h8 (stateless, input=8) | 140.10        | 35.69        | +1.1%        |

No variant outperforms the baseline. The results are compromised by the transpose bug (Section 3.1) and the encoding path difference. Even without these issues, the stateless design provides no mechanism for cross-rollout memory.

### 5.3 Phase 2: Baseline Reference (Fair Comparison Grid)

Table 3: Phase 2 baselines: mesh\_size=4, batch\_size=1, 20k steps completed

| input_steps | target_steps | RMSE @20k    | MAE @20k     |
|-------------|--------------|--------------|--------------|
| 2           | 2            | 62.41        | 18.21        |
| <b>4</b>    | <b>2</b>     | <b>59.59</b> | <b>17.48</b> |
| 2           | 4            | 96.08        | 26.89        |
| 4           | 4            | 95.13        | 26.71        |

Key observations:

- **Longer input history helps:** input\_steps=4 outperforms input\_steps=2 by  $\sim 5\%$  RMSE, confirming that the model benefits from more temporal context in the input window.

- **Longer rollout training is harder:** `target_steps=4` produces substantially worse metrics than `target_steps=2` ( $\sim 50\%$  higher RMSE), because the cumulative autoregressive loss is a more difficult optimization target.

#### 5.4 Phase 2: Stateful Mamba (Old Encoding Path)

Table 4: Phase 2 stateful Mamba: `mesh_size=4`, `batch_size=1`, old encoding path (cancelled at  $\sim 70\%$  completion)

| Config          | Best RMSE | Eval Step | Baseline @20k | Gap    |
|-----------------|-----------|-----------|---------------|--------|
| sf-h2, target=2 | 72.70     | @14k      | 62.41         | +16.5% |
| sf-h4, target=2 | 71.37     | @12k      | 59.59         | +19.8% |
| sf-h2, target=4 | 108.41    | @12k      | 96.08         | +12.8% |
| sf-h4, target=4 | 111.09    | @10k      | 95.13         | +16.8% |

All stateful Mamba configurations trail the baseline by 13–20%. While loss was still decreasing at cancellation, the gap was too large to close within the remaining training budget. These experiments were cancelled because:

1. The encoding path difference confounds interpretation—we cannot determine whether Mamba helps or hurts.
2. The per-timestep encoding path is likely weaker than channel concatenation, masking any potential benefit from temporal memory.
3. A new architecture (residual memory) was designed to address these issues.

## 6 Diagnosis: Why Previous Approaches Failed

Table 5: Summary of identified problems across architectural versions

| Problem                       | Description                                                               | Impact                                                                                                                            |
|-------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Transpose bug (V1)            | Mesh and batch dimensions swapped in output of <code>_run_sequence</code> | All V1 Mamba outputs had corrupted dimension ordering; model trained to baseline-like loss only because GNN partially compensated |
| Encoding path change (V1, V2) | Per-timestep encoding instead of channel concatenation                    | Weaker encoder; confounded comparison with baseline; cannot attribute results to Mamba                                            |
| Redundant processing (V1, V2) | Mamba processes 2–8 frames that channel concatenation already handles     | Extra parameters and optimization cost with no new information                                                                    |
| Short rollout (V2)            | <code>target_steps = 2</code> or <code>4</code>                           | Insufficient sequential depth for hidden state to accumulate useful long-term information                                         |

The central insight is that GraphCast’s channel concatenation is already an effective temporal encoder for the input window. Mamba’s value should come from *cross-rollout* memory—



information that the 2-frame input window cannot provide—not from re-processing the input frames.

## 7 Phase 3: Residual Memory Experiments

### 7.1 Experimental Design

Six controlled experiments, differing only in (a) whether the Mamba residual block is present and (b) the number of autoregressive rollout steps during training:

Table 6: Phase 3 experiment matrix (500 steps smoke test, mesh\_size=4, batch\_size=1, input\_steps=2)

|                     | target_steps=2 | target_steps=4 | target_steps=6 |
|---------------------|----------------|----------------|----------------|
| Baseline (no Mamba) | baseline_t2    | baseline_t4    | baseline_t6    |
| Residual Memory     | resmem_t2      | resmem_t4      | resmem_t6      |

### 7.2 Interpretation Guide

- **resmem.tX vs. baseline.tX:** Isolates the Mamba contribution at each rollout length. If resmem matches or beats baseline, temporal memory is providing value.
- **Trend across target\_steps:** If Mamba’s advantage grows with target\_steps, this confirms that longer rollouts unlock more benefit from temporal memory.
- **Minimum bar:** Due to the residual connection, resmem should perform no worse than baseline. If it does, this indicates an optimization issue (e.g., Mamba parameters interfering with gradient flow).

### 7.3 Status

These experiments have been submitted (SLURM job array 6527518) and are pending GPU allocation as of April 4, 2026.

## 8 Discussion

### 8.1 Why Residual Memory Is the Right Formulation

The evolution from V1 to V3 reflects a progressive understanding of where Mamba can add value in the GraphCast pipeline:

- **V1 (stateless):** Mamba as a *replacement* for channel-concatenation temporal encoding → redundant and weaker.
- **V2 (stateful, old encoding):** Mamba with cross-rollout state but *still replacing* the encoding → correct idea, wrong execution.
- **V3 (residual memory):** Mamba as a *supplement* to the existing encoding, providing *only* cross-rollout information → minimal intervention, maximal isolation.

### 8.2 Open Questions

1. **Capacity:** The Mamba hidden state is 128-dimensional, the same as the mesh latent. Is this sufficient to encode a useful compressed trajectory summary? Larger hidden sizes (512, 1024) may be needed.

2. **Placement:** The current Mamba block is placed after the Grid-to-Mesh encoder, before the Mesh GNN. An alternative placement after the Mesh GNN (before decoding) would allow Mamba to operate on higher-level, spatially-processed features.
3. **Long-rollout inference:** Even if Mamba shows no benefit at training-time rollout lengths, it may help during inference when the model is rolled out for 20–40 steps (5–10 day forecasts). Evaluating existing checkpoints on long rollouts could reveal benefits invisible in short-rollout metrics.
4. **Single-frame input:** With `input_steps=1` (no velocity information from frame differencing), temporal memory becomes the *only* source of cross-timestep information. This could amplify Mamba’s contribution.

### 8.3 Computational Considerations

Table 7: Estimated training time per 10k steps on A100 80GB

| Configuration                                 | Time per step | Total (10k steps) |
|-----------------------------------------------|---------------|-------------------|
| Baseline, <code>target_steps=2</code>         | ~3s           | ~8h               |
| Residual memory, <code>target_steps=2</code>  | ~3.5s         | ~10h              |
| Residual memory, <code>target_steps=6</code>  | ~10s          | ~28h              |
| Residual memory, <code>target_steps=10</code> | ~15s          | ~42h              |

The residual memory architecture adds minimal overhead per rollout step (~15% for `target_steps=2`). The dominant cost factor is `target_steps`, which linearly scales the forward/backward pass cost regardless of whether Mamba is used.

## 9 Conclusion

We identified three critical issues in our initial Mamba integration into GraphCast: a dimension transpose bug, a confounded encoding path, and redundant temporal processing. The corrected *residual memory* architecture addresses all three by preserving the baseline encoding path and injecting temporal memory solely through a stateful residual block.

Whether temporal memory ultimately improves weather forecasting at this model scale remains an open question. However, the residual memory architecture provides the cleanest possible experiment to answer it: any performance difference can be attributed entirely to cross-rollout temporal memory, with no confounding factors.

## 10 Why Mamba May Not Surpass the Baseline

Beyond the bugs and design flaws already identified, there are deeper reasons why adding temporal memory to GraphCast may be fundamentally difficult at this scale:

### 10.1 The Baseline Is Already Surprisingly Effective

GraphCast’s channel concatenation encodes  $[\mathbf{x}_{t-1}; \mathbf{x}_t]$  as a  $2C$ -dimensional vector, which an MLP can trivially decompose into:

- Current state:  $\mathbf{x}_t$
- First-order tendency:  $\mathbf{x}_t - \mathbf{x}_{t-1}$  (velocity)
- Any nonlinear combination of the two frames

With `input_steps=4`, the model additionally has access to second-order tendencies (acceleration) and longer-range trends. Our experiments confirm that `input_steps=4` reduces RMSE by 5% over `input_steps=2`. This means the 4-frame input window already captures the most salient temporal information for 6h prediction, leaving little room for an SSM to add value.

## 10.2 Information Bottleneck: 128 Dimensions for an Entire Atmosphere

The Mamba hidden state  $h \in \mathbb{R}^{128}$  per mesh node must compress the *entire history of atmospheric evolution* at that spatial location. The mesh latent itself is 128-dimensional and encodes the current atmospheric state. Asking a 128-dimensional hidden state to carry additional information *beyond* what the current state already provides is a severe information bottleneck.

For context, at `mesh_size=4` there are 2,562 mesh nodes, each with 128-dimensional hidden state, giving  $\sim 328\text{k}$  total hidden state parameters. The atmospheric state being represented has  $\sim 90$  grid points  $\times$  180 grid points  $\times$  (6 atmospheric variables  $\times$  13 levels + 5 surface variables)  $\approx 1.4\text{M}$  scalar values. The hidden state cannot represent a meaningful fraction of the trajectory history.

## 10.3 The Forgetting Problem

The SSM decay  $\bar{A}_t = \exp(A \cdot \Delta_t)$  controls how quickly the hidden state forgets. With a single layer and 128 dimensions, the model faces a dilemma:

- **Fast decay** ( $\bar{A} \approx 0$ ): The state effectively resets each step, degenerating to stateless behavior.
- **Slow decay** ( $\bar{A} \approx 1$ ): The state becomes a running average, diluting recent information with stale history.
- **Selective decay**: The intended design—but with only 128 dimensions, the model cannot simultaneously maintain both fast-changing and slow-changing information channels.

## 10.4 Optimization Difficulty of Long Rollouts

Even with gradient checkpointing, training through  $K$  autoregressive steps requires gradients to flow through  $K$  sequential forward passes. This creates:

- **Gradient vanishing/exploding**: Through  $K$  multiplications of the SSM decay matrix and the GNN’s nonlinearities.
- **Credit assignment**: The loss at step  $K$  must be attributed to hidden state updates at steps  $1, 2, \dots, K-1$ . This is a long-range credit assignment problem that is notoriously difficult.
- **Training instability**: Our results show that `target_steps=4` produces  $\sim 50\%$  worse RMSE than `target_steps=2` for the *baseline* (no Mamba), confirming that longer rollout training is intrinsically harder.

## 10.5 The Core Problem: Marginal Value

Fundamentally, the question is whether the atmospheric state at time  $t$  contains insufficient information to predict  $t+6\text{h}$ , such that information from  $t-6\text{h}, t-12\text{h}, \dots$  would materially help. For most weather phenomena at 6h resolution, the answer is likely *no*—the current state and its recent tendency encode enough physics to make a good short-range prediction. The exceptions (slowly evolving blocking patterns, MJO propagation, etc.) may be too rare in the training data to drive a consistent learning signal.

The marginal value of temporal memory is highest when:

- The input window is very short (1 frame, no velocity information)
- The rollout is very long (10+ days, where error drift dominates)
- The model capacity is large enough to learn meaningful state representations

- The training data contains enough long-range temporal structure to learn from  
None of these conditions are strongly met in our current experimental setup.

## References

- [1] R. Lam et al., “Learning skillful medium-range global weather forecasting,” *Science*, vol. 382, no. 6677, pp. 1416–1421, 2023.
- [2] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.